

3-SAT

Dokumentacja końcowa

Algorytmy zaawansowane

Klaudia Buczek, Klara Pluta, Gabriela Warchoł

1 Zmiany względem dokumentacji wstępnej

Dokumentacja wstępna przedstawiała podstawową wersję aplikacji, która przyjmuje formułę w postaci 3-CNF, szuka rozwiązania jednocześnie ją upraszczając. Na koniec zwraca informację, czy jest ona spełnialna oraz wyświetla rozwiązanie, jeżeli ono istnieje. Ostateczna wersja aplikacji nie różni się znacznie od tego, co było przedstawione w dokumentacji wstępnej. Z dodatkowych funkcjonalności pojawił się m.in. generator pliku wejściowego. Jeżeli aplikacja nie widzi pliku wejściowego, to sama generuje taki plik dla losowej liczby zmiennych ($n \in \langle 5, 40 \rangle$) i losowej liczby klauzul ($m \in \langle 10, 200 \rangle$). Program zwraca również czas, jaki był potrzebny na znalezienie rozwiązania.

2 Opis programu dla użytkownika

Aplikacja zawiera menu tekstowe, które krok po kroku pokazuje użytkownikowi, co ma zrobić. Uruchomienie skryptu odbywa się za pomocą standardowej komendy w terminalu:

```
python 3SAT.py
```

po uruchomieniu programu użytkownik ma do wyboru trzy opcje:

- Rozwiąż formułę z pliku - program szuka pliku "przyklad.txt" w katalogu, w którym znajduje się skrypt. Jeżeli plik istnieje, to program go wczytuje i próbuje znaleźć rozwiązanie. Jeżeli plik nie istnieje, to program generuje losową formułę i zapisuje ją do tego pliku, a następnie próbuje znaleźć rozwiązanie.
- Wygeneruj nową losową formułę (podaj własne parametry) - użytkownik jest proszony o podanie liczby zmiennych (n) oraz liczby klauzul (m). Program generuje losową formułę 3-CNF zgodnie z podanymi parametrami i zapisuje ją do pliku "przyklad.txt", a następnie próbuje znaleźć rozwiązanie.
- Wygeneruj nową losową formułę (losowe parametry) - program losowo wybiera liczbę zmiennych ($n \in \langle 5, 40 \rangle$) oraz liczbę klauzul ($m \in \langle 10, 200 \rangle$), generuje losową formułę 3-CNF zgodnie z tymi parametrami i zapisuje ją do pliku "przyklad.txt", a następnie próbuje znaleźć rozwiązanie.

2.1 Format danych wejściowych i wyjściowych

Format plików pozostał zgodny z założeniami projektowymi. W pliku wejściowym pierwsza linia określa odpowiednio liczbę zmiennych oraz liczbę klauzul. Każda kolejna linia odpowiada jednej klauzuli, gdzie liczba dodatnia n oznacza n -tą zmienną, a liczba ujemna $-n$ jej negację.

Plik wyjściowy zawiera czytelny nagłówek ze statusem (ROZWIĄZALNE / NIEROZWIĄZALNE), przypisanie wartości logicznych (0 lub 1) do poszczególnych zmiennych (w przypadku powodzenia) oraz dokładny czas, jaki program potrzebował na zakończenie obliczeń.

3 Testy i wnioski

W ramach testów zbadano czas działania algorytmu dla różnej liczby zmiennych (n) oraz różnej liczby klauzul (m). Dla każdego zestawu parametrów (n, m) przeprowadzono 10 testów, a następnie obliczono średni czas działania algorytmu. Wyniki zostały przedstawione w tabeli poniżej. Wyniki jednoznacznie potwierdzają wykładniczy charakter problemu 3-SAT. Zwiększenie liczby

Liczba zmiennych (n)	Liczba klauzul (m)	Średni czas [s]
10	30	0,0010
10	43	0,0011
10	50	0,0015
20	60	0,0020
20	90	0,0311
20	100	0,0525
30	90	0,0116
30	130	0,3748
30	150	0,3436
35	110	0,0171
35	150	0,5048
35	180	0,6942
40	120	0,0424
40	170	2,4492
40	200	2,1728

Tabela 1: Wyniki testów wydajnościowych dla wybranych parametrów n oraz m .

zmiennych ma najbardziej znaczący wpływ na czas obliczeń. Dodatkowo przy niskim stosunku liczby klauzul do liczby zmiennych (m/n) czas działania jest znacznie krótszy, co jest zgodne z oczekiwaniami, ponieważ formuły o niskim stosunku m/n są zazwyczaj łatwiejsze do rozwiązania. W miarę zwiększania liczby klauzul, czas działania rośnie, co również jest zgodne z teorią, ponieważ większa liczba klauzul zwiększa złożoność problemu.

Analiza danych dla $n = 30$ oraz $n = 40$ pokazuje, że czas działania algorytmu nie rośnie liniowo wraz z liczbą klauzul. Od pewnego momentu czas ten się skraca, wynika to z tego, że przy dużej liczbie klauzul prostsze jest upraszczanie poprzez usuwanie sprzeczności.

Podczas testów nie odnotowano awarii programu ani błędnych wyników. Algorytm poprawnie identyfikował status "NIEROZWIĄZYWALNE" dla formuł, które nie mają rozwiązania, oraz zwracał poprawne rozwiązania dla formuł spełnialnych.

Na podstawie przeprowadzonych testów oraz analizy teoretycznej złożoności obliczeniowej, przyj-

mując kryterium akceptowalnego czasu oczekiwania na poziomie około 10 sekund, jako ostateczną rozsądną granicę wydajności zaimplementowanej aplikacji przyjęto $n = 50$ **zmiennych**.

4 Podział prac

- Klaudia Buczek: testy, dokumentacja końcowa.
- Klara Pluta: implementacja generatora instancji, dokumentacja wstępna.
- Gabriela Warchoł: przygotowanie wersji beta aplikacji